

Programação Web

Spring Data JPA, Entities e Repository

Prof. Dr. Gabriel Gadelha

UNIVERSIDADE FEDERAL RURAL DO SEMI-ÁRIDO

Agenda

- Revisão da aula anterior
- Spring Data JPA
- Jakarta EE
- Entities
- Repository
- Teste

TodoController

```
@RestController
@RequestMapping(🌐✓"/api/v1/users/{userId}/todos")
public class TodoController {
    @GetMapping(🌐✓)
    public ResponseEntity<List<TodoResponse>> listar(
        @PathVariable Long userId) {
        return null;}

    @GetMapping(🌐✓"/{todoId}")
    public ResponseEntity<TodoResponse> buscarPorId(
        @PathVariable Long userId,
        @PathVariable Long todoId) {
        return null;}
```

TodoController

@PostMapping 🌐

```
public ResponseEntity<TodoResponse> criar(  
    @PathVariable Long userId,  
    @RequestBody TodoCreate dto,  
    UriComponentsBuilder uriBuilder) {  
    return null;}  

```

@PutMapping(🌐 "/{todoId}")

```
public ResponseEntity<TodoResponse> atualizar(  
    @PathVariable Long userId,  
    @PathVariable Long todoId,  
    @RequestBody TodoUpdate dto) {  
    return null;}  

```

TodoController

```
@PatchMapping(🌐📄"/{todoId}")
public ResponseEntity<TodoResponse> alterarParcial(
    @PathVariable Long userId,
    @PathVariable Long todoId,
    @RequestBody TodoPatch dto) {
    return null;
}

@DeleteMapping(🌐📄"/{todoId}")
public ResponseEntity<Void> remover(
    @PathVariable Long userId,
    @PathVariable Long todoId) {
    return null;}
}
```

Organização da aplicação

 java

▼  br.edu.ufersa.pw.todo.todoAPI

▼  api

>  controllers

>  dtos

▼  domain

 entities

 repositories

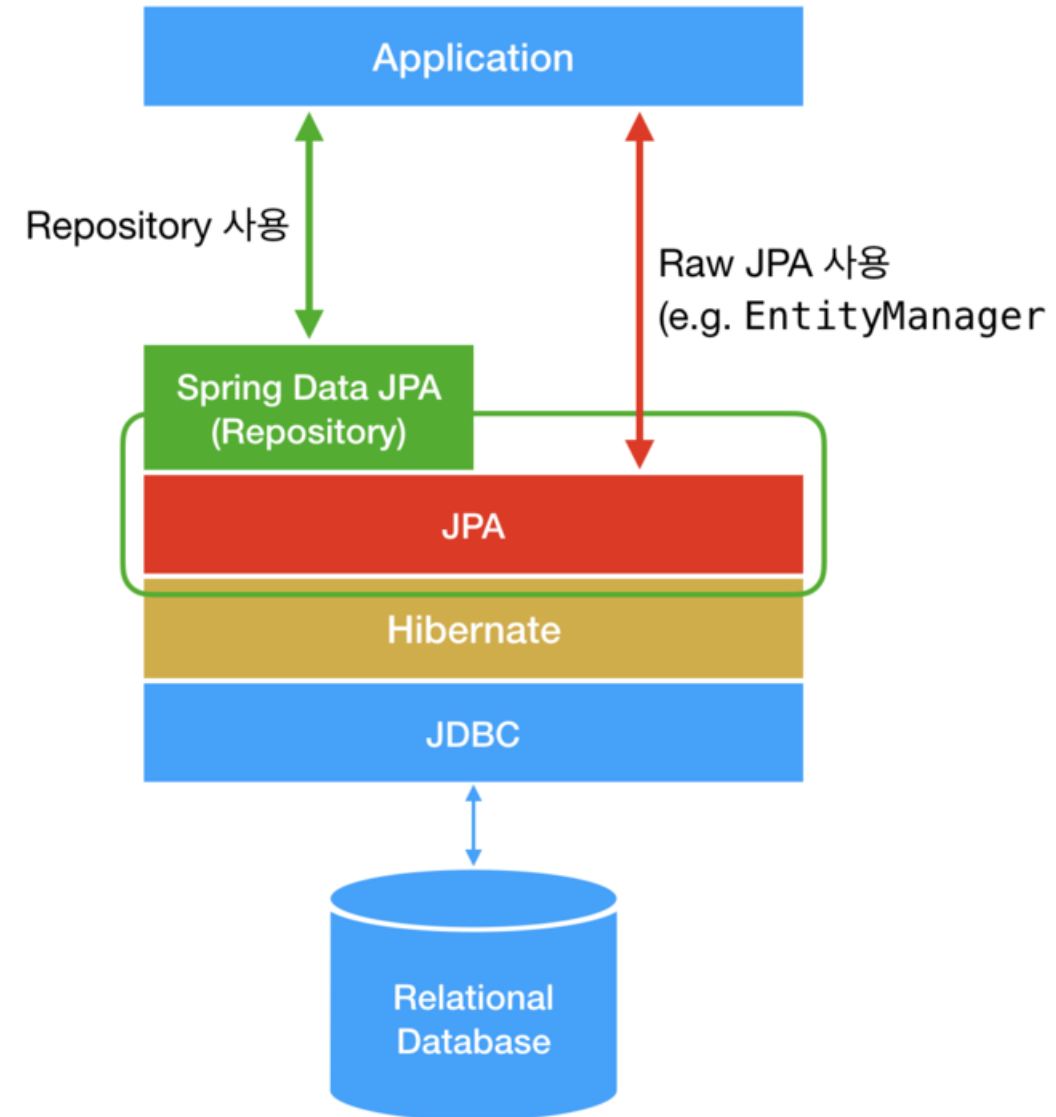
 TodoApiApplication

Spring Data JPA

- O que é?
 - Subprojeto do Spring
 - Facilita a implementação de Repositórios JPA
 - Aplicações com tecnologias de acesso a dados (BD)
- JPA
 - Java Persistence API
 - Mapeamento Objeto-Relacional (ORM - Object-Relational Mapping)
 - Hibernate e outros
 - Jakarta Persistence

Repositório JPA

- É uma interface que simplifica o acesso ao banco de dados em aplicações Java, eliminando a necessidade de escrever comandos SQL manuais.



Um pouco de História

- J2EE
 - 1999 (1.2) – 2005 (1.4)
 - J2EE Platform – uma plataforma padrão, que segue as especificações de um conjunto de APIs para a hospedagem de aplicativos J2EE – EJB, JSP, JDBC, JNDI, JTA ...
 - Web Container (Glassfish) e EJB Container
 - J2EE Reference Implementation
 - Sun Blueprints Design Guidelines for J2EE
 - JCP (Java Community Process)
 - TCK (Technology Compatibility Kit)

Um pouco de História

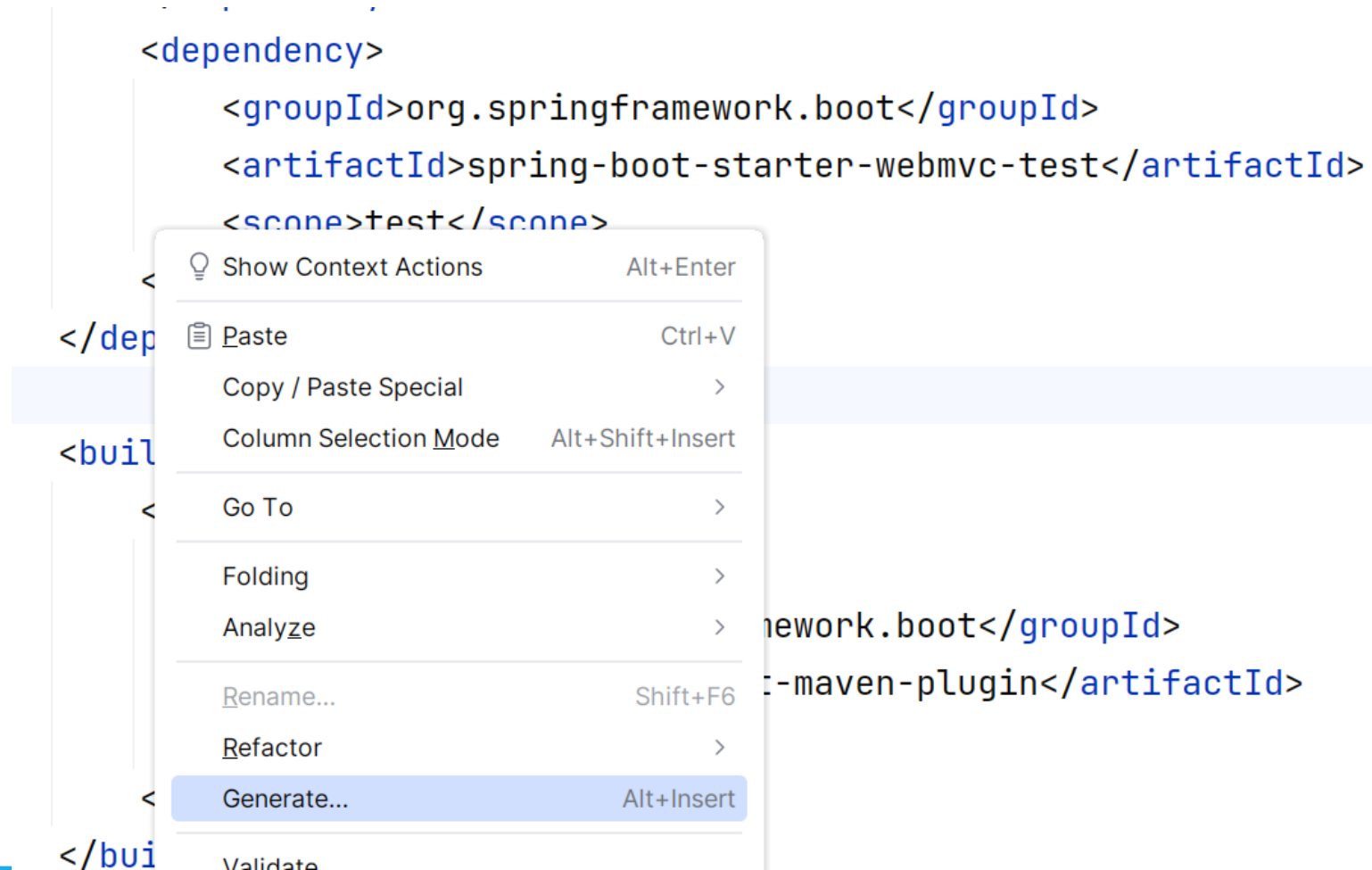
- Java EE (Oracle)
 - 2006 (Java EE 5) – 2017 (Java EE 8)
 - JCP passou a ser comandado dentro da Oracle
 - Interesse crescente em Nuvem e redução do esforço na evolução do Java EE
 - Java EE estava ficando para trás
 - 2017 → doação dos direitos da especificação Java EE a **Eclipse Foundation**
 - **ENTRETANTO:** Oracle mantém os direitos sobre o nome Java EE

Um pouco de História

- Projeto Jakarta
 - Foi um importante projeto elaborado pela Apache Foundation
 - Apache Tomcat, Struts, Commons, Velocity, Maven
 - Descontinuado em 2011 – Projetos separados
 - O nome Jakarta foi sugerido como substituição ao Java EE e doado pela Apache Foundation a Eclipse Foundation
 - **Jakarta EE** (Jakarta Eclipse Enterprise)
 - O objetivo é manter o Java corporativo sempre atual em relação as tendências e demandas do mercado
 - Jakarta Persistence 3.0 (**JPA**)

Importar Dependências

Botão da direita dentro do pom.xml → Generate → Add Starters → spring data jpa



Importar Dependências

× Add Starters

Server URL: start.spring.io 

Spring Boot: 4.1.1

Dependencies:

 spring data jpa ×

SQL

☒ Spring Data JPA

Spring Data JPA

Persist data in SQL stores with Java Persistence API using Spring Data and Hibernate.

[Accessing Data with JPA ↗](#)

Added dependencies:

Importar Dependências

✕

Add Starters

Server URL: start.spring.io ⚙

Spring Boot: 4.1.1

Dependencies:

🔍 mysql ✕

▼ SQL

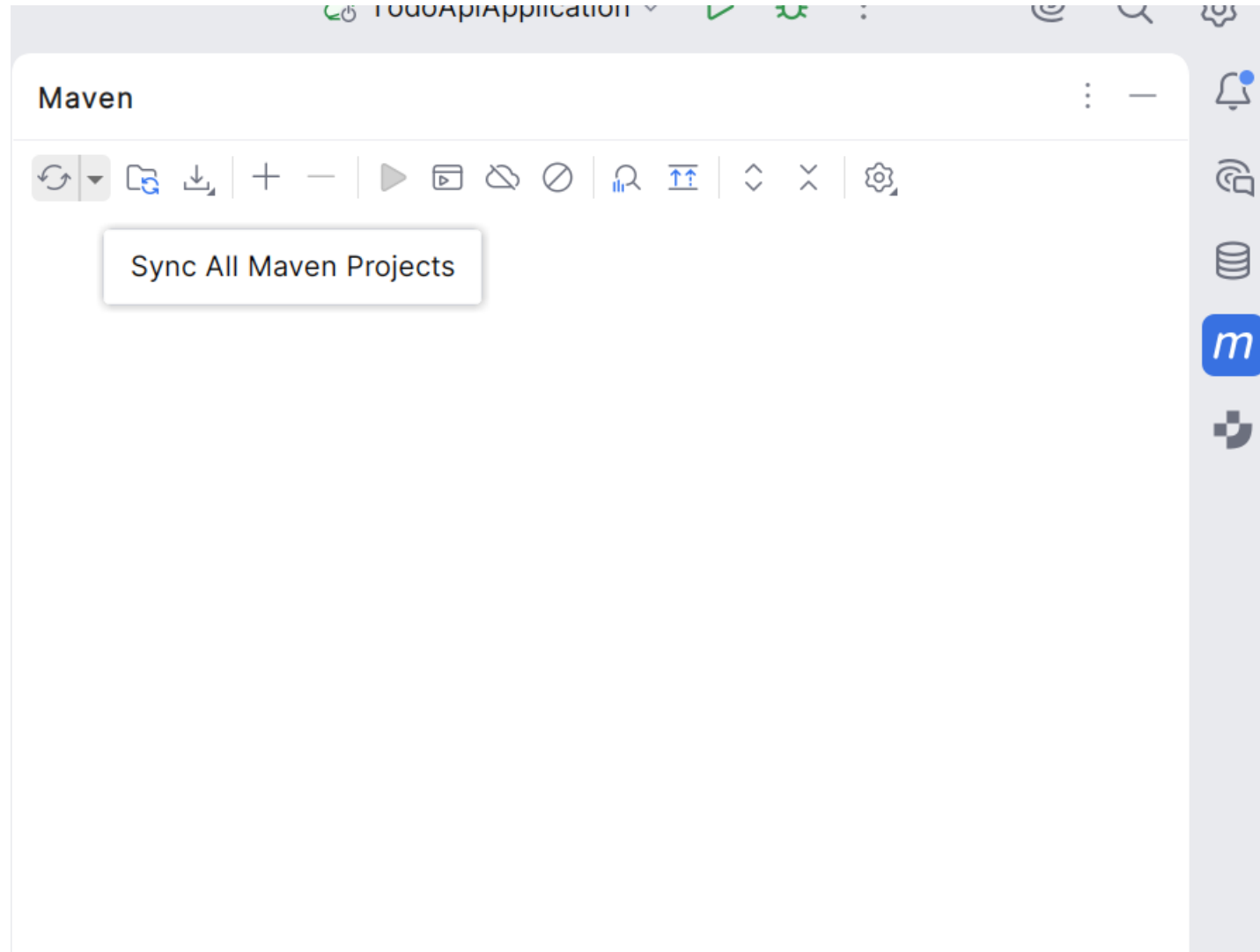
☒ MySQL Driver

MySQL Driver

MySQL JDBC driver.

[Accessing data with MySQL ↗](#)

Importar Dependências



Configurar BD no application.properties

```
spring.application.name=todoAPI
var1=valor definido no app.properties
spring.datasource.url=jdbc:mysql://localhost:3306/todo?createDatabaseIfNotExist=TRUE
spring.datasource.username=pw
spring.datasource.password=M3lhormateria!
spring.jpa.show-sql=true
spring.jpa.hibernate.ddl-auto=update
```


Criar as domain.entities

- ✓  domain
 - ✓  entities
 -  Email
 -  Estado
 -  Senha
 -  Todo
 -  Usuario

Criar a entity Todo

```
public class Todo { 2 usages
    private final Long id; 2 usages
    private final Usuario usu; 2 usages
    private String item; 3 usages
    private LocalDate prazo; 5 usages
    private Estado estado; 9 usages
    private LocalDate conclusao; 3 usages
```

Criar a entity Todo

```
private Todo(Builder builder) { 1 usage
    this.id = builder.id;
    this.usu = builder.usu;
    this.item = builder.item;
    this.prazo = builder.prazo;
    this.estado = builder.estado;
    this.conclusao = builder.conclusao;
}
```

```
public void concluir() { no usages
    validarTransicao(Estado.CONCLUIDO);
    this.estado = Estado.CONCLUIDO;
    this.conclusao = LocalDate.now();
}
```

Criar a entity Todo

```
public void indicarAtraso() { no usages
    validarTransicao(Estado.ATRASADO);
    this.estado = Estado.ATRASADO;
}
```

```
public void prorrogarPrazo(LocalDate novoPrazo) { no usages
    if (this.estado == Estado.CONCLUIDO) {
        throw new IllegalStateException("Não é possível alterar o prazo de tarefas concluídas.");
    }
    Objects.requireNonNull(novoPrazo, message: "O novo prazo é obrigatório.");
    if (this.prazo != null && novoPrazo.isBefore(LocalDate.now())) {
        throw new IllegalArgumentException("O novo prazo não pode ser anterior à data de hoje.");
    }
    if (this.prazo != null && novoPrazo.isBefore(this.prazo)) {
        throw new IllegalArgumentException("O novo prazo não pode antecipar o prazo já vigente.");
    }
    this.prazo = novoPrazo;
    this.estado = Estado.EM_ANDAMENTO;
}
```

Criar a entity Todo

```
public void renomearItem(String novoItem) { no usages
    if (this.estado == Estado.CONCLUIDO) {
        throw new IllegalStateException("Não é permitido renomear tarefas
    }
    if (novoItem == null || novoItem.isBlank()) {
        throw new IllegalArgumentException("A descrição do item não pode
    }
    this.item = novoItem;
}
```

```
private void validarTransicao(Estado proximoEstado) { 2 usages
    if (!this.estado.podeTransicionarPara(proximoEstado)) {
        throw new IllegalStateException(
            String.format("Transição inválida: tarefa está em '%s' e
                this.estado, proximoEstado)
        );
    }
}
```

Criar a entity Todo

```
public Long getId() { return id; } no usages
public Usuario getUsu() { return usu; } no usages
public String getItem() { return item; } no usages
public LocalDate getPrazo() { return prazo; } no usages
public Estado getEstado() { return estado; } no usages
public LocalDate getConclusao() { return conclusao; }
```

Criar a entity Todo

```
public static class Builder { 5 usages
    // Obrigatórios
    private final Usuario usu; 2 usages
    private final String item; 2 usages

    // Opcionais com valores padrão explícitos
    private Long id; // ATENÇÃO AQUI 2 usages
    private LocalDate prazo = LocalDate.now(); 4
    private Estado estado = Estado.EM_ANDAMENTO;
    private LocalDate conclusao; 6 usages
```

Criar a entity Todo

```
public Builder(Usuario usu, String item) { no usages
    this.usu = Objects.requireNonNull(usu, message: "O usuário é obrigatório!");
    if (item == null || item.isBlank()) {
        throw new IllegalArgumentException("O item é obrigatório!");
    }
    this.item = item;
}
```

```
public Builder comId(Long id) { no usages
    this.id = id;
    return this;
}
```


Criar a entity Todo

```
public Builder comPrazo(LocalDate prazo) { no usages
    this.prazo = prazo;
    return this;
}
```

```
public Builder comEstado(Estado estado) { no usages
    this.estado = estado;
    return this;
}
```

```
public Builder comConclusao(LocalDate conclusao) { no usages
    this.conclusao = conclusao;
    return this;
}
```

Criar a entity Todo

```
public Todo build() { no usages
    validarInvariantes();
    return new Todo(builder: this);
}
```

```
private void validarInvariantes() { 1 usage
    if (this.conclusao != null && this.conclusao.isBefore(LocalDate.now())) {
        throw new IllegalArgumentException("A data de conclusão não pode ser ant
    }
    if (this.prazo != null && this.conclusao != null && this.conclusao.isBefore(
        throw new IllegalArgumentException("A conclusão não pode ser anterior a
    }
}
```

Criar a entity Todo

```
public enum Estado { 14 usages
    EM_ANDAMENTO, 5 usages
    CONCLUIDO, 7 usages
    ATRASADO; 3 usages
    public boolean podeTransicionarPara(Estado novoEstado) { 1 usage
        return switch (this) {
            case EM_ANDAMENTO -> Set.of(CONCLUIDO, ATRASADO).contains(novoEstado);
            case CONCLUIDO -> false; // Estado terminal
            case ATRASADO -> Set.of(EM_ANDAMENTO, CONCLUIDO).contains(novoEstado);
        };
    }
}
```

Anotar a classe Todo

```
@Entity
@Table(name="tb_todos")
public class Todo {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    @ManyToOne(fetch = FetchType.LAZY, optional = false)
    @JoinColumn(name = "id_user", nullable = false)
    private Usuario usu;
    @Column(nullable = false) 4 usages
```

Anotar a classe Todo

```
@Column(nullable = false) 4 usages
private String item;
@Column 5 usages
private LocalDate prazo;
@Enumerated(EnumType.STRING) 9 usages
@Column
private Estado estado;
@Column 3 usages
private LocalDate conclusao;

protected Todo() {
}
```

Criar equals e hash

@Override

```
public boolean equals(Object o) {  
    if (this == o) return true;  
    if (o == null || getClass() != o.getClass()) return false;  
    Todo todo = (Todo) o;  
    return id != null && id.equals(todo.id);  
}
```

@Override

```
public int hashCode() {  
    return Objects.hash(usu, item);  
}
```

Criar Interface TodoRepository

```
@Repository //opcional neste caso. Me perguntem pq 3 usages
public interface TodoRepository extends JpaRepository<Todo, Long> {
    List<Todo> findByUsuId(Long userId); 1 usage
    Optional<Todo> findByIdAndUsuId(Long id, Long userId); no usages
}
```

Injetar UserRepository p/ uso

```
@RestController
@RequestMapping(🌐✓"/api/v1/users/{userId}/todos")
public class TodoController {
    private final TodoRepository todoRepository; 3 usages

    public TodoController(TodoRepository todoRepository) {
        this.todoRepository = todoRepository;
    }
}
```


Métodos do TodoController

```
public TodoController(TodoRepository todoRepository) { this.todoRepository = todoRepository; }  
@GetMapping() 🌐  
public ResponseEntity<List<TodoResponse>> listar(  
    @PathVariable Long userId) {  
    List<TodoResponse> lista = todoRepository.findByUsuId(userId) List<Todo>  
        .stream() Stream<Todo>  
        .map(TodoResponse::fromEntity) Stream<TodoResponse>  
        .toList();  
    return ResponseEntity.ok(lista);  
}
```

Métodos do TodoController

@PostMapping🌐

```
public ResponseEntity<TodoResponse> criar(
    @PathVariable Long userId,
    @RequestBody TodoCreate dto,
    UriComponentsBuilder uriBuilder) {
    Todo.Builder builder = new Todo.Builder(new Usuario(userId), dto.item());
    if (dto.prazo() != null) { builder.comPrazo(dto.prazo()); }
    if(dto.estado() !=null){ builder.comEstado(dto.estado().toDomain());}
    Todo novoTodo = builder.build();
    Todo salvo = todoRepository.save(novoTodo);
    URI uri = uriBuilder
        .path("/api/v1/users/{userId}/todos/{todoId}") UriComponentsBuilder
        .buildAndExpand(userId, salvo.getId()) UriComponents
        .toUri();
    return ResponseEntity.created(uri).body(TodoResponse.fromEntity(salvo));
}
```

Insomnia

The screenshot displays the Insomnia REST client interface. The top bar includes a search bar, a 'Personal Workspace' dropdown, and an 'Invite' button. The left sidebar shows a project tree with 'My first project' and 'Testes_Todo'. The main panel shows a GET request to 'http://localhost:8080/api/v1/users/1/todos'. The response is a 200 OK status with a response time of 81 ms and a body size of 324 B. The response body is a JSON array of three todo items, each with an id, item description, due date (prazo), and status (estado).

Personal Workspace ▾

Search.. Ctrl + P

Invite

Essentials Plan ▾

Projects (1)

Filter + New Project

My first project

Testes_Todo

GET Listar

POST Add Todos

POST Add Todos × GET Listar × +

My first project > Testes_Todo > GET Listar

Base Environment Cookies (1) Certificate

GET http://localhost:8080/api/v1/users/1/todos Send 200 OK 81 ms 324 B 2 Minutes Ago ▾

Params Body Auth Headers 3 Scripts Docs

No Body ▾

Preview Headers 3 Cookies Tests 0/0 → Mock Console

```
1 [
2   {
3     "id": 1,
4     "item": "Preparar slides da aula de Spring Data JPA",
5     "prazo": "2026-09-03",
6     "estado": "EM_ANDAMENTO"
7   },
8   {
9     "id": 2,
10    "item": "Publicar um vídeo sobre projeto de api no youtube",
11    "prazo": "2026-09-04",
12    "estado": "EM_ANDAMENTO"
13  },
14  {
15    "id": 3,
16    "item": "Postar a primeira parte do projeto no sigaa",
17    "prazo": "2026-09-02",
18    "estado": "ATRASADO"
19  }
20 ]
```

Enter a URL and send to get a response

Select a body type from above to send data in the body of a request

Próxima Aula

Main Class	@SpringBootApplication	Spring Boot auto configuration
REST Endpoint	@RestController	Class with REST endpoints
	@RequestMapping	REST endpoint method
	@PathVariable	URI path parameter
	@RequestBody	HTTP request body
Periodic Tasks	@Scheduled	Method to run periodically
	@EnableScheduling	Enable Spring's task scheduling
Beans	@Configuration	A class containing Spring beans
	@Bean	Objects to be used by Spring IoC for dependency injection
Spring Managed Components	@Component	A candidate for dependency injection
	@Service	Like @Component
	@Repository	Like @Component, for data base access
Persistence	@Entity	A class which can be stored in the data base via ORM
	@Id	Primary key
	@GeneratedValue	Generation strategy of primary key
	@EnableJpaRepositories	Triggers the search for classes with @Repository annotation
	@EnableTransactionManagement	Enable Spring's DB transaction management through @Beans objects
Miscellaneous	@Autowired	Force dependency injection
	@ConfigurationProperties	Import settings from properties file
Testing	@SpringBootTest	Spring integration test
	@AutoConfigureMockMvc	Configure MockMvc object to test HTTP queries

Domain Service vs Application Service

Domain Service



Characteristics:

- Pure business logic
- No infrastructure dependencies
- Coordinates multiple entities
- Testable in isolation
- Lives in Domain layer

```
MoneyTransferManager
Transfer(from, to, amount)
```

Application Service



Characteristics:

- Orchestrates use cases
- Handles repositories
- Manages transactions
- Calls domain services
- Lives in Application layer

```
BankAppService
TransferAsync(fromId, toId, amount)
```

VS